Unit 2. Linear Models and Regularization

Business Data Analytics: Introductory Machine Learning

1 The OLS Setup

1.1 Linear-algebra preliminaries

Definition 2.1 (Linear-algebra vocabulary)**.** A nonempty $V \subseteq \mathbb{R}^n$ is a **subspace** if it is closed under linear combinations (so $0 \in V$, $u+v \in V$, $\alpha u \in V$ for all $u, v \in V, \alpha \in \mathbb{R}$). For $A \in \mathbb{R}^{m\times n}$, viewed as the linear map $\mathbb{R}^n \to \mathbb{R}^m$, $v \mapsto Av$:

Concept	Definition
Span of $v_1, \dots, v_k$	$\{\sum_i c_i v_i : c_i \in \mathbb{R}\}$; always a subspace
Linear independence	$\sum_i c_i v_i = 0 \Rightarrow c_i = 0$ for every i
Basis of V	linearly independent spanning set
Dimension $\dim V$	cardinality of any basis (independent of choice)
Image $\mathrm{img}(A) = \mathrm{col}(A)$	$\{Av : v \in \mathbb{R}^n\}$ = span of columns; subspace of $\mathbb{R}^m$
Kernel $\ker(A)$	$\{v \in \mathbb{R}^n : Av = 0\}$; subspace of $\mathbb{R}^n$
Rank $\mathrm{rank}(A)$	$\dim \mathrm{img}(A)$
Nullity $\mathrm{null}(A)$	$\dim \ker(A)$
Orthogonal vectors	$u \perp v \Leftrightarrow u^\top v = 0$
Orthogonal subspaces	$U \perp V \Leftrightarrow u^\top v = 0$ for all $u \in U, v \in V$
Orthogonal complement $V^\perp$	$\{u : u \perp v \ \forall v \in V\}$; subspace of $\mathbb{R}^n$
Orthogonal direct sum	$\mathbb{R}^n = V \oplus V^\perp$; every $x = v + u$ uniquely, $\dim V + \dim V^\perp = n$

Theorem 2.2 (Rank–nullity)**.** Let $A \in \mathbb{R}^{m\times n}$, viewed as a linear map $\mathbb{R}^n \to \mathbb{R}^m$. Then

$$n \;=\; \operatorname{rank}(A) + \operatorname{null}(A),$$

where $\operatorname{rank}(A) := \dim \operatorname{img}(A)$ and $\operatorname{null}(A) := \dim \ker(A)$.

Proof. Let $k = \operatorname{null}(A)$ and fix a basis $\{v_1, \dots, v_k\}$ of $\ker(A)$; extend to a basis $\{v_1, \dots, v_k, u_1, \dots, u_{n-k}\}$ of $\mathbb{R}^n$. It suffices to show that $\{Au_1, \dots, Au_{n-k}\}$ is a basis of $\operatorname{img}(A)$: then $\operatorname{rank}(A) = n - k$, giving $n = k + (n-k) = \operatorname{null}(A) + \operatorname{rank}(A)$.

Spanning. For any $w = Av \in \operatorname{img}(A)$, write $v = \sum_i c_i v_i + \sum_j d_j u_j$. Each $v_i \in \ker(A)$, so $Av_i = 0$, and

$$Av \;=\; \sum_i c_i \cdot 0 + \sum_j d_j Au_j \;=\; \sum_j d_j Au_j.$$

Hence $\{Au_j\}$ spans $\operatorname{img}(A)$.

Linear independence. Suppose $\sum_j d_j Au_j = 0$. Then $A\big(\sum_j d_j u_j\big) = 0$, so $\sum_j d_j u_j \in \ker(A)$,

which we expand in the kernel basis as $\sum_j d_j u_j = \sum_i c_i v_i$ for some c_i. Rearranging,

$$\sum_j d_j u_j \;-\; \sum_i c_i v_i \;=\; 0.$$

Linear independence of the extended basis $\{v_i\} \cup \{u_j\}$ forces every $c_i = 0$ and every $d_j = 0$; the $\{Au_j\}$ are therefore linearly independent. □

Definition 2.3 (Matrix types)**.** For $A \in \mathbb{R}^{p\times p}$:

Concept	**Definition**
Symmetric matrix	$A^\top = A$
Diagonal matrix $\mathrm{diag}(\lambda_1, \ldots, \lambda_p)$	off-diagonal entries zero; (i,i)-entry equal to λ_i
Orthogonal matrix Q	square with $Q^\top Q = QQ^\top = I_p$ (columns orthonormal)
Positive semidefinite $A \succcurlyeq 0$	symmetric A with $v^\top Av \geqslant 0$ for every $v \in \mathbb{R}^p$
Positive definite $A \succ 0$	symmetric A with $v^\top Av > 0$ for every $v \in \mathbb{R}^p, v \neq 0$

Lemma 2.4 (Left inverse equals right inverse for square matrices)**.** For square $Q \in \mathbb{R}^{p\times p}$, if $Q^\top Q = I_p$ then Q is invertible with $Q^{-1} = Q^\top$; in particular, $QQ^\top = I_p$. (So defining an orthogonal matrix by orthonormal columns automatically gives orthonormal rows.)

Proof. $Q^\top Q = I_p$ implies $\ker(Q)$ is trivial: if $Qv = 0$, then $v = Q^\top Qv = Q^\top \cdot 0 = 0$. By Theorem 2.2, $\operatorname{rank}(Q) = p - \operatorname{null}(Q) = p$. Since Q is $p\times p$ with rank p, $Q : \mathbb{R}^p \to \mathbb{R}^p$ is bijective, so Q^{-1} exists. Right-multiplying $Q^\top Q = I_p$ by Q^{-1} gives $Q^\top = Q^{-1}$; hence $QQ^\top = QQ^{-1} = I_p$. □

Definition 2.5 (Eigenvalue and eigenvector)**.** For symmetric $A \in \mathbb{R}^{p\times p}$, $\lambda \in \mathbb{R}$ is an **eigenvalue** with **eigenvector** $v \in \mathbb{R}^p, v \neq 0$, if $Av = \lambda v$. (Reality of every eigenvalue of a real symmetric A is part of the spectral theorem proven below.)

Theorem 2.6 (Spectral theorem)**.** Every symmetric $A \in \mathbb{R}^{p\times p}$ admits an **orthogonal diagonalization**

$$A = Q\Lambda Q^\top, \qquad \Lambda = \operatorname{diag}(\lambda_1, \ldots, \lambda_p),\ Q^\top Q = QQ^\top = I_p,$$

with real eigenvalues λ_j and orthonormal eigenvectors $q_1, \ldots, q_p$ (the columns of Q). When $A \succ 0$,

all $\lambda_j > 0$ and $A^{-1} = Q\,\mathrm{diag}(\lambda_j^{-1})Q^\top$.

Proof. We construct $q_1, \ldots, q_p$ iteratively as solutions of constrained Rayleigh-quotient maximizations. Set $W_1 := \mathbb{R}^p$; for $j = 1, 2, \ldots, p$, define

$$q_j \;:=\; \arg\max_{x \in W_j,\ \|x\|=1} x^\top A x, \qquad \lambda_j \;:=\; q_j^\top A q_j, \qquad W_{j+1} \;:=\; W_j \cap \{q_j\}^\perp.$$

Step 1: q_j exists. $\{x \in W_j : \|x\| = 1\}$ is closed and bounded in finite-dimensional $\mathbb{R}^p$, hence compact (Heine–Borel); $x \mapsto x^\top A x$ is continuous, so by the Weierstrass extreme value theorem the maximum is attained.

Step 2: q_j is an eigenvector with eigenvalue λ_j. Unfolding the recursion $W_{j+1} = W_j \cap \{q_j\}^\perp$ starting from $W_1 = \mathbb{R}^p$ gives

$$W_j \;=\; \mathrm{span}(q_1, \ldots, q_{j-1})^\perp, \qquad W_{j+1} \;=\; \mathrm{span}(q_1, \ldots, q_j)^\perp,$$

so $W_j^\perp = \mathrm{span}(q_1, \ldots, q_{j-1})$ and $W_{j+1}^\perp = \mathrm{span}(q_1, \ldots, q_j)$.

For any direction $w \in W_{j+1}$ (so $w \in W_j$ and $w \perp q_j$), the curve

$$x(t) \;:=\; \frac{q_j + tw}{\|q_j + tw\|}, \quad t \in (-\delta, \delta) \text{ small},$$

stays in W_j (linear combinations of $q_j, w \in W_j$) and on the unit sphere by construction, so the function $g(t) := x(t)^\top A x(t)$ attains its maximum at $t = 0$. Hence $g'(0) = 0$. Using $\|q_j\| = 1$, $q_j \perp w$, and $A^\top = A$, expand the numerator and denominator:

$$(q_j + tw)^\top A(q_j + tw) = q_j^\top A q_j + 2t(Aq_j)^\top w + t^2 w^\top A w \;=\; \lambda_j + 2t(Aq_j)^\top w + O(t^2),$$
$$\|q_j + tw\|^2 = 1 + t^2\|w\|^2.$$

Their ratio gives $g(t) = \lambda_j + 2t(Aq_j)^\top w + O(t^2)$, so

$$0 \;=\; g'(0) \;=\; 2(Aq_j)^\top w.$$

Since this holds for every $w \in W_{j+1}$, $Aq_j \in W_{j+1}^\perp = \operatorname{span}(q_1, \ldots, q_j)$. Write

$$Aq_j \;=\; \lambda\, q_j + \sum_{i=1}^{j-1} \alpha_i\, q_i.$$

To eliminate the α_k for $k < j$, take the inner product of both sides with q_k:

$$q_k^\top A q_j \;=\; \lambda\,(q_k^\top q_j) + \sum_i \alpha_i\,(q_k^\top q_i) \;=\; \alpha_k,$$

using orthonormality of $(q_1, \ldots, q_j)$. Independently, the previous iterations of the construction already produced $Aq_k = \lambda_k q_k$, so

$$q_k^\top A q_j \;=\; (Aq_k)^\top q_j \;=\; \lambda_k\,(q_k^\top q_j) \;=\; 0,$$

using $A^\top = A$. Hence $\alpha_k = 0$ for every $k < j$, and $Aq_j = \lambda q_j$. Taking the inner product with q_j gives $\lambda = q_j^\top A q_j = \lambda_j$, which is real since $q_j^\top A q_j$ is a real expression.

Step 3: $(q_1, \ldots, q_p)$ is an orthonormal basis of $\mathbb{R}^p$. $\|q_j\| = 1$ by construction; $q_j \perp q_i$ for every $i < j$ by the constraint defining W_j. The q_j are pairwise orthogonal unit vectors, and any p such vectors in $\mathbb{R}^p$ form a basis.

Step 4: Assemble. Set $Q := [q_1\ q_2\ \cdots\ q_p]$ and $\Lambda := \mathrm{diag}(\lambda_1, \ldots, \lambda_p)$. Orthonormality of the columns gives $Q^\top Q = QQ^\top = I_p$; the eigenvalue equations $Aq_j = \lambda_j q_j$ $(j = 1, \ldots, p)$ read $AQ = Q\Lambda$, and right-multiplying by $Q^\top$ yields $A = Q\Lambda Q^\top$.

PSD case. If $A \succ 0$, then for each unit eigenvector q_j, $\lambda_j = q_j^\top A q_j > 0$; $\Lambda^{-1} = \operatorname{diag}(\lambda_j^{-1})$ exists, and $(Q\Lambda Q^\top)(Q\Lambda^{-1}Q^\top) = Q\Lambda\Lambda^{-1}Q^\top = QQ^\top = I_p$, so $A^{-1} = Q\Lambda^{-1}Q^\top$. □

The construction in the proof of Theorem 2.6 defines each q_j as the argmax of $x^\top Ax$ over $\{x \in W_j : \|x\| = 1\}$. Reading this off as a numbered statement gives the Rayleigh-quotient characterization of every eigenvalue, used in Unit 7 §7.2 (PCA variance maximization) and the Eckart–Young proof.

Theorem 2.7 (Rayleigh-quotient maxima)**.** Let $A \in \mathbb{R}^{p\times p}$ be symmetric with eigenvalues $\lambda_1 \geqslant \cdots \geqslant \lambda_p$ and orthonormal eigenvectors $q_1, \ldots, q_p$ as in Theorem 2.6. Then

$$\lambda_1 \;=\; \max_{x \neq \mathbf{0}} \frac{x^\top Ax}{x^\top x} \;=\; \max_{\|x\|=1} x^\top Ax,$$

attained at $x = q_1$. For $k = 2, \ldots, p$,

$$\lambda_k \;=\; \max_{\substack{\|x\|=1 \\ x \perp q_1, \ldots, q_{k-1}}} x^\top Ax,$$

attained at $x = q_k$. Symmetrically, λ_p is the minimum of $x^\top Ax$ on the unit sphere, attained at

q_p.

Proof. Expand a unit vector x in the basis $q_1, \dots, q_p$: $x = \sum_j c_j q_j$ with $\sum_j c_j^2 = 1$. Using $Aq_j = \lambda_j q_j$ and orthonormality,

$$x^\top A x \;=\; \sum_{j=1}^{p} c_j^2 \lambda_j,$$

a convex combination of $\lambda_1, \dots, \lambda_p$, hence in $[\lambda_p, \lambda_1]$, with maximum λ_1 at $c_1 = 1$, $c_2 = \cdots = c_p = 0$, i.e. $x = q_1$. Homogeneity of the Rayleigh quotient $x^\top Ax / x^\top x$ gives the unconstrained form. The constraint $x \perp q_1, \dots, q_{k-1}$ forces $c_1 = \cdots = c_{k-1} = 0$, so $x^\top Ax = \sum_{j \geqslant k} c_j^2 \lambda_j \leqslant \lambda_k$, attained at $x = q_k$. The minimum statement follows by applying the maximum to $-A$ (eigenvalues $-\lambda_p \geqslant \cdots \geqslant -\lambda_1$). □

1.2 Matrix model and classical assumptions

Definition 2.8 (Linear regression model and Gauss–Markov assumptions)**.** The **linear regression model** is

$$y = X\beta + \varepsilon, \qquad y \in \mathbb{R}^n,\ X \in \mathbb{R}^{n\times p},\ \beta \in \mathbb{R}^p,\ \varepsilon \in \mathbb{R}^n,$$

with X's first column typically $\mathbf{1}_n$ for an intercept. Linearity is in β; columns of X may be nonlinear transforms (powers, logs, interactions) of raw features. The classical Gauss–Markov framework adds:

ID	Statement	Yields
A1	**Linearity**: $y = X\beta + \varepsilon$ for fixed β	the target β is well-defined
A2	**Full column rank**: $\operatorname{rank}(X) = p$	$\hat\beta = (X^\top X)^{-1}X^\top y$ is unique
A3	**Exogeneity**: $\mathsf{E}[\varepsilon \mid X] = 0$	$\mathsf{E}[\hat\beta \mid X] = \beta$ (unbiased)
A4	**Spherical errors**: $\operatorname{var}(\varepsilon \mid X) = \sigma^2 I_n$	$\hat\beta$ is BLUE (Gauss–Markov)

1.3 Normal equations and the OLS estimator

OLS minimizes the squared-error objective

$$Q(\beta) \;=\; \|y - X\beta\|^2 \;=\; (y - X\beta)^\top (y - X\beta).$$

Lemma 2.9 (Rank of $X^\top X$)**.** For any $X \in \mathbb{R}^{n\times p}$, $\operatorname{rank}(X^\top X) = \operatorname{rank}(X)$. Under A2, $X^\top X$ is therefore invertible.

Proof. X and $X^\top X$ share their null space: $Xv = 0 \Rightarrow X^\top Xv = 0$ by left-multiplying by $X^\top$; conversely $X^\top Xv = 0 \Rightarrow v^\top X^\top Xv = \|Xv\|^2 = 0 \Rightarrow Xv = 0$. Both $X : \mathbb{R}^p \to \mathbb{R}^n$ and $X^\top X : \mathbb{R}^p \to \mathbb{R}^p$ have input dimension p and the same nullity, so Theorem 2.2 applied to each gives

$$\operatorname{rank}(X) \;=\; p - \operatorname{null}(X) \;=\; p - \operatorname{null}(X^\top X) \;=\; \operatorname{rank}(X^\top X).$$

Under A2, $\operatorname{rank}(X) = p$; hence $X^\top X$ is a full-rank $p \times p$ matrix and is invertible. □

Lemma 2.10 (Gradients of linear and quadratic forms)**.** For $\beta \in \mathbb{R}^p$, constant $a \in \mathbb{R}^p$, and

symmetric $A \in \mathbb{R}^{p\times p}$, with $D(\cdot)$ denoting the row-vector gradient ($Df = (\partial f/\partial\beta_1, \ldots, \partial f/\partial\beta_p)$):

$$D(a^\top\beta) \;=\; a^\top, \qquad D(\beta^\top A\beta) \;=\; 2\beta^\top A.$$

Proof. For the linear form, $a^\top\beta = \sum_i a_i\beta_i$, so $\partial(a^\top\beta)/\partial\beta_k = a_k$, giving $D(a^\top\beta) = a^\top$. For the quadratic form, $\beta^\top A\beta = \sum_{i,j} A_{ij}\,\beta_i\beta_j$. Differentiate term by term:

$$\frac{\partial(\beta^\top A\beta)}{\partial\beta_k} \;=\; \sum_j A_{kj}\beta_j + \sum_i A_{ik}\beta_i \;=\; \sum_j (A_{kj}+A_{jk})\beta_j \;=\; 2\sum_j A_{kj}\beta_j \;=\; 2(A\beta)_k,$$

using $A_{jk} = A_{kj}$ (symmetry). Hence $D(\beta^\top A\beta) = 2(A\beta)^\top = 2\beta^\top A$ (again by symmetry). □

Theorem 2.11 (OLS estimator)**.** Under A2, Q has a unique minimizer

$$\hat\beta \;=\; (X^\top X)^{-1}X^\top y.$$

Proof. Step 1: Expand. Using $y^\top X\beta = \beta^\top X^\top y$ (a scalar equals its transpose),

$$Q(\beta) \;=\; y^\top y - 2\beta^\top X^\top y + \beta^\top X^\top X\beta.$$

Step 2: First-order condition. Apply Lemma 2.10 with $a = X^\top y$ and $A = X^\top X$:

$$DQ(\beta) \;=\; -2\,y^\top X + 2\,\beta^\top X^\top X.$$

Setting $DQ(\beta) = 0$ and transposing gives the **normal equation**

$$X^\top X\hat{\beta} \;=\; X^\top y.$$

Step 3: Solve. By Lemma 2.9 and A2, $X^\top X$ is invertible, so

$$\hat{\beta} \;=\; (X^\top X)^{-1}X^\top y.$$

Step 4: Second-order condition. The Hessian $\nabla^2 Q = 2X^\top X$. For any $v \neq 0$, under A2, $v^\top X^\top Xv = \|Xv\|^2 > 0$, so $X^\top X \succ 0$. Hence Q is strictly convex on $\mathbb{R}^p$ and $\hat{\beta}$ is the unique global minimum. □

1.4 Geometric reading: orthogonal projection

Under A2, $\dim \operatorname{col}(X) = \operatorname{rank}(X) = p$, so OLS searches for the closest point in a p-dimensional subspace of $\mathbb{R}^n$. The resulting fit has a clean geometric description.

Theorem 2.12 (Orthogonality characterization of OLS)**.** For $\hat{y} \in \operatorname{col}(X)$, the following are equivalent:

(i) $\hat{y}$ minimizes $\|y - \tilde{y}\|^2$ over $\tilde{y} \in \operatorname{col}(X)$;

(ii) $y - \hat{y} \perp \operatorname{col}(X)$, i.e. $v^\top (y - \hat{y}) = 0$ for every $v \in \operatorname{col}(X)$;

(iii) $X^\top (y - \hat{y}) = 0$.

The minimizer is unique under A2 and equals $\hat{y} = X\hat{\beta}$ with $\hat{\beta} = (X^\top X)^{-1} X^\top y$.

Proof. **(ii) $\Leftrightarrow$ (iii).** The columns of X span $\operatorname{col}(X)$, so $y - \hat{y}$ is orthogonal to every $v \in \operatorname{col}(X)$ iff orthogonal to each column iff $X^\top (y - \hat{y}) = 0$.

(ii) $\Rightarrow$ (i). Let $\tilde{y} \in \operatorname{col}(X)$. Since $\operatorname{col}(X)$ is a subspace, $\hat{y} - \tilde{y} \in \operatorname{col}(X)$, so $(y - \hat{y}) \perp (\hat{y} - \tilde{y})$. Pythagoras applied to $y - \tilde{y} = (y - \hat{y}) + (\hat{y} - \tilde{y})$ gives

$$\|y - \tilde{y}\|^2 \;=\; \|y - \hat{y}\|^2 + \|\hat{y} - \tilde{y}\|^2 \;\geqslant\; \|y - \hat{y}\|^2,$$

with equality iff $\tilde{y} = \hat{y}$.

(i) $\Rightarrow$ (ii). Suppose $\hat{y}$ minimizes but some $v \in \mathrm{col}(X)$ has $v^\top(y - \hat{y}) \neq 0$; rescale to $\|v\| = 1$. For $\tilde{y}(t) = \hat{y} + tv \in \mathrm{col}(X)$,

$$\|y - \tilde{y}(t)\|^2 \;=\; \|y - \hat{y}\|^2 - 2t\, v^\top(y - \hat{y}) + t^2,$$

minimized at $t^\star = v^\top(y - \hat{y}) \neq 0$ with value $\|y - \hat{y}\|^2 - (t^\star)^2 < \|y - \hat{y}\|^2$, contradicting (i).

Closed form. Writing $\hat{y} = X\hat{\beta}$, condition (iii) becomes $X^\top X\hat{\beta} = X^\top y$, the normal equation of §1.3. Under A2, $\hat{\beta} = (X^\top X)^{-1}X^\top y$, and since the orthogonal projection onto a closed subspace is unique, so is $\hat{y}$. □

Definition 2.13 (Hat matrix)**.** The **hat matrix** (orthogonal projector onto $\mathrm{col}(X)$) is

$$P_X \;:=\; X(X^\top X)^{-1}X^\top \;\in\; \mathbb{R}^{n\times n},$$

so that $\hat{y} = P_X y$.

P_X has three structural properties characterizing an orthogonal projection:

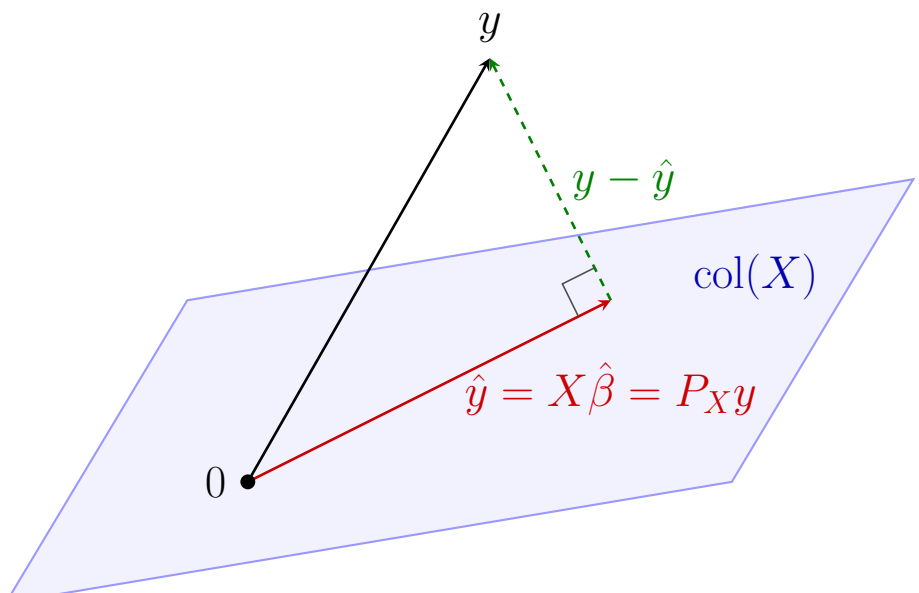


Figure 2.1: Orthogonal projection of y onto $\mathrm{col}(X)$: The residual $y - \hat{y}$ is orthogonal to every vector in $\mathrm{col}(X)$, yielding the normal equations $X^\top(y - X\hat{\beta}) = 0$; equivalently $X^\top(y - \hat{y}) = 0$ (Theorem 2.12 (iii)).

Property	Statement	Verification
Symmetric	$P_X^\top = P_X$	$(X(X^\top X)^{-1}X^\top)^\top = X(X^\top X)^{-1}X^\top$
Idempotent	$P_X^2 = P_X$	$X(X^\top X)^{-1}(X^\top X)(X^\top X)^{-1}X^\top = X(X^\top X)^{-1}X^\top$
Range	$\mathrm{img}(P_X) = \mathrm{col}(X)$	$P_X(Xb) = Xb$ for every $b \in \mathbb{R}^p$; $P_X y \in \mathrm{col}(X)$ by inspection

Symmetry uses $((X^\top X)^{-1})^\top = (X^\top X)^{-1}$, since the inverse of a symmetric matrix is symmetric. Idempotence collapses the middle factor $(X^\top X)(X^\top X)^{-1}$ to I_p.

1.5 Ill-conditioning: variance blow-up and the case for regularization

Lemma 2.14 (Linear transformation of variance)**.** For a random vector $z \in \mathbb{R}^n$ with finite second moments and any constant matrix $B \in \mathbb{R}^{m\times n}$, $\mathrm{var}(Bz) = B\ \mathrm{var}(z)\, B^\top$. Conditionally, if B is a function of variables in the conditioning set,

$$\mathrm{var}(Bz \mid X) \;=\; B\ \mathrm{var}(z \mid X)\, B^\top.$$

Proof. By definition, $\operatorname{var}(Bz) = \mathsf{E}\big[(Bz - \mathsf{E}[Bz])(Bz - \mathsf{E}[Bz])^\top\big]$. Linearity of expectation gives $\mathsf{E}[Bz] = B\,\mathsf{E}[z]$, hence $Bz - \mathsf{E}[Bz] = B(z - \mathsf{E}\,z)$. Substituting,

$$\operatorname{var}(Bz) \;=\; \mathsf{E}\big[B(z-\mathsf{E}\,z)(z-\mathsf{E}\,z)^\top B^\top\big] \;=\; B\;\mathsf{E}\big[(z-\mathsf{E}\,z)(z-\mathsf{E}\,z)^\top\big]\,B^\top \;=\; B\;\operatorname{var}(z)\,B^\top,$$

pulling B and $B^\top$ out of the (linear) expectation. The conditional version is identical with $\mathsf{E}[\cdot \mid X]$ throughout, since B is constant given X. □

Theorem 2.15 (Variance of OLS)**.** Under A1–A4, conditional on X,

$$\operatorname{var}(\hat\beta \mid X) \;=\; \sigma^2(X^\top X)^{-1}.$$

Proof. Set $M := (X^\top X)^{-1}X^\top$. Substitute $y = X\beta + \varepsilon$:

$$\hat\beta \;=\; My \;=\; MX\beta + M\varepsilon \;=\; \beta + M\varepsilon,$$

using $MX = (X^\top X)^{-1}X^\top X = I_p$. Conditional on X, M is constant; by Lemma 2.14 and A4

$(\mathrm{var}(\varepsilon \mid X) = \sigma^2 I_n)$,

$$\mathrm{var}(\hat{\beta} \mid X) \;=\; \mathrm{var}(M\varepsilon \mid X) \;=\; M(\sigma^2 I_n)M^\top \;=\; \sigma^2 MM^\top.$$

Compute $MM^\top = (X^\top X)^{-1}X^\top X(X^\top X)^{-1} = (X^\top X)^{-1}$. □

Definition 2.16 (Condition number)**.** For symmetric $A \succ 0$ with eigenvalues $\lambda_1 \geqslant \cdots \geqslant \lambda_p > 0$, the **condition number** is $\kappa(A) := \lambda_1/\lambda_p$. κ is large when some eigenvalue is much smaller than the largest.

Apply Theorem 2.6 to $X^\top X$ with eigenvalues $\lambda_1 \geqslant \cdots \geqslant \lambda_p$ and orthonormal eigenvectors q_j. By Theorem 2.15,

$$\mathrm{var}(\hat{\beta} \mid X) \;=\; \sigma^2(X^\top X)^{-1} \;=\; \sigma^2 \sum_{j=1}^{p} \lambda_j^{-1} q_j q_j^\top.$$

The variance along the j-th principal direction is σ^2/λ_j. When λ_p is small, the variance along q_p explodes; this is the algebraic face of **near-collinearity** in the columns of X.

Failure mode	Algebraic signature	OLS consequence
Near-collinearity	$\lambda_p \to 0^+$, high $\kappa(X^\top X)$	some $\mathrm{var}(\hat\beta_j) \to \infty$
Exact collinearity	$\lambda_p = 0$	$X^\top X$ singular; A2 fails; $\hat\beta$ undefined
$p > n$	$\lambda_{n+1} = \cdots = \lambda_p = 0$	A2 fails; $\hat\beta$ undefined

All three share the same algebraic root: an eigenvalue of $X^\top X$ is too small or zero. Regularization addresses this by perturbing $X^\top X$ so its eigenvalues are bounded away from zero.

2 Ridge Regression

2.1 The ridge problem and closed-form solution

Definition 2.17 (Ridge regression)**.** For a tuning parameter $\lambda \geqslant 0$, the **ridge regression** estimator minimizes the average squared-error loss with an ℓ_2 penalty on β:

$$\hat\beta_\lambda \;:=\; \arg\min_{\beta\in\mathbb{R}^p} R_\lambda(\beta), \qquad R_\lambda(\beta) \;:=\; \tfrac{1}{2n}\|y - X\beta\|^2 + \tfrac{\lambda}{2}\|\beta\|^2.$$

At $\lambda = 0$ this recovers OLS; for $\lambda > 0$ the penalty pulls $\hat\beta_\lambda$ toward the origin, trading bias for

variance. The factor $1/(2n)$ follows the modern ML convention (`glmnet`, scikit-learn): it makes λ approximately invariant to sample size and lets the formulas below match production solvers without rescaling.

Lemma 2.18 (Invertibility of $X^\top X + n\lambda I_p$)**.** For any $X \in \mathbb{R}^{n\times p}$ and any $\lambda > 0$, $X^\top X + n\lambda I_p$ is positive definite, hence invertible. No assumption on $\operatorname{rank}(X)$ is needed; the ridge estimator is well-defined even when A2 fails.

Proof. For any $v \in \mathbb{R}^p$ with $v \neq 0$,

$$v^\top (X^\top X + n\lambda I_p)\, v \;=\; \|Xv\|^2 + n\lambda \|v\|^2 \;\geqslant\; n\lambda \|v\|^2 \;>\; 0,$$

so $X^\top X + n\lambda I_p \succ 0$. By Theorem 2.6, all its eigenvalues are positive, and the inverse exists in spectral form. □

Theorem 2.19 (Closed-form ridge estimator)**.** For any $\lambda > 0$, the unique minimizer of R_λ on

$\mathbb{R}^p$ is

$$\hat{\beta}_\lambda \;=\; (X^\top X + n\lambda I_p)^{-1} X^\top y.$$

Proof. Expand R_λ:

$$R_\lambda(\beta) \;=\; \tfrac{1}{2n}\big(y^\top y - 2\beta^\top X^\top y + \beta^\top X^\top X\beta\big) + \tfrac{\lambda}{2}\beta^\top\beta.$$

By Lemma 2.10, the row-vector gradient is

$$DR_\lambda(\beta) \;=\; -\tfrac{1}{n}\, y^\top X + \tfrac{1}{n}\,\beta^\top X^\top X + \lambda\,\beta^\top \;=\; \tfrac{1}{n}\,\beta^\top(X^\top X + n\lambda I_p) - \tfrac{1}{n}\, y^\top X.$$

Setting $DR_\lambda(\beta) = 0$ and transposing gives the **ridge normal equation**

$$(X^\top X + n\lambda I_p)\,\hat{\beta}_\lambda \;=\; X^\top y.$$

By Lemma 2.18, the coefficient matrix is invertible, so $\hat{\beta}_\lambda = (X^\top X + n\lambda I_p)^{-1}X^\top y$. The Hessian $\nabla^2 R_\lambda = \frac{1}{n}(X^\top X + n\lambda I_p) \succ 0$, so R_λ is strictly convex and $\hat{\beta}_\lambda$ is the unique global minimum. □

When $X^\top X$ has eigenvalues $\sigma_1^2 \geqslant \cdots \geqslant \sigma_p^2$ (the squared singular values of X), the perturbed matrix

$X^\top X + n\lambda I_p$ has eigenvalues $\sigma_j^2 + n\lambda$. The ridge condition number drops from $\kappa(X^\top X) = \sigma_1^2/\sigma_p^2$ to

$$\kappa(X^\top X + n\lambda I_p) \;=\; \frac{\sigma_1^2 + n\lambda}{\sigma_p^2 + n\lambda},$$

which is much smaller when $\sigma_p^2 \ll n\lambda$: the penalty term lifts the spectrum away from zero, remedying the failures catalogued in §1.5.

2.2 Principal-component shrinkage

By Theorem 2.6 applied to the PSD matrix $X^\top X$, there exist orthonormal eigenvectors $v_1, \dots, v_p \in \mathbb{R}^p$ (the **right singular vectors** of X) and eigenvalues $\lambda_1 \geqslant \cdots \geqslant \lambda_p \geqslant 0$, giving

$$X^\top X \;=\; V \Lambda V^\top, \qquad V := [v_1 \;\cdots\; v_p],\; \Lambda := \operatorname{diag}(\lambda_1, \dots, \lambda_p).$$

The **singular values** of X are $\sigma_j := \sqrt{\lambda_j} \geqslant 0$ (so $\sigma_1 \geqslant \cdots \geqslant \sigma_p \geqslant 0$). For indices with $\sigma_j > 0$, define the corresponding **left singular vectors** in $\mathbb{R}^n$ by

$$u_j \;:=\; \frac{X v_j}{\sigma_j}.$$

Lemma 2.20 (Orthonormality of left singular vectors)**.** For indices j, k with $\sigma_j, \sigma_k > 0$, $u_j^\top u_k = \delta_{jk}$. $Xv_j = \sigma_j u_j$ for those j, and $Xv_j = 0$ when $\sigma_j = 0$.

Proof. $u_j^\top u_k = (Xv_j)^\top (Xv_k)/(\sigma_j\sigma_k) = v_j^\top X^\top X v_k/(\sigma_j\sigma_k) = \lambda_k\,(v_j^\top v_k)/(\sigma_j\sigma_k) = \delta_{jk}$ on simplifying $\lambda_k v_j^\top v_k = \sigma_k^2\delta_{jk}$ and $\sigma_j\sigma_k = \sigma_k^2$ when $j = k$. The relation $Xv_j = \sigma_j u_j$ for $\sigma_j > 0$ is the definition. When $\sigma_j = 0$, $\|Xv_j\|^2 = v_j^\top X^\top X v_j = \lambda_j = 0$, so $Xv_j = 0$. □

Theorem 2.21 (Ridge as principal-component shrinkage)**.** The ridge estimator and fitted values decompose as

$$\hat\beta_\lambda \;=\; \sum_{j:\sigma_j>0} \frac{\sigma_j(u_j^\top y)}{\sigma_j^2 + n\lambda}\, v_j, \qquad \hat y_\lambda \;:=\; X\hat\beta_\lambda \;=\; \sum_{j:\sigma_j>0} \frac{\sigma_j^2}{\sigma_j^2 + n\lambda}\,(u_j^\top y)\, u_j.$$

Each **shrinkage factor** $\sigma_j^2/(\sigma_j^2 + n\lambda) \in [0, 1)$ for $\lambda > 0$, approaching 1 as $\sigma_j^2 \to \infty$ and 0 as $\sigma_j^2 \to 0$.

Proof. Since $VV^\top = I_p$, $X^\top X + n\lambda I_p = V\Lambda V^\top + n\lambda VV^\top = V(\Lambda + n\lambda I_p)V^\top$, and inverting (Lemma 2.4) gives $(X^\top X + n\lambda I_p)^{-1} = V(\Lambda + n\lambda I_p)^{-1}V^\top$. Therefore

$$\hat\beta_\lambda \;=\; (X^\top X + n\lambda I_p)^{-1}X^\top y \;=\; V(\Lambda + n\lambda I_p)^{-1}V^\top X^\top y.$$

The j-th coordinate of $V^\top\hat\beta_\lambda$ is

$$v_j^\top\hat\beta_\lambda \;=\; \frac{v_j^\top X^\top y}{\sigma_j^2 + n\lambda} \;=\; \frac{(Xv_j)^\top y}{\sigma_j^2 + n\lambda} \;=\; \begin{cases} \sigma_j(u_j^\top y)/(\sigma_j^2 + n\lambda) & \text{if } \sigma_j > 0, \\ 0 & \text{if } \sigma_j = 0, \end{cases}$$

using $Xv_j = \sigma_j u_j$ (Lemma 2.20). Reassembling $\hat\beta_\lambda = \sum_j (v_j^\top\hat\beta_\lambda)v_j$ gives the first formula; substituting into $\hat y_\lambda = X\hat\beta_\lambda$ and applying $Xv_j = \sigma_j u_j$ gives the second. □

Interpretation. At $\lambda = 0$ under A2, $\hat y = \sum_j (u_j^\top y)\, u_j$ is the orthogonal projection of y onto $\mathrm{col}(X) = \mathrm{span}(u_1, \ldots, u_p)$. Ridge with $\lambda > 0$ damps each principal direction u_j by $\sigma_j^2/(\sigma_j^2 + n\lambda)$: high-signal directions ($\sigma_j^2 \gg n\lambda$) pass nearly unchanged, while low-signal directions ($\sigma_j^2 \ll n\lambda$), precisely those exposed by ill-conditioning in §1.5, are damped almost to zero.

2.3 Effective degrees of freedom

OLS fits exactly p parameters; ridge constrains all of them simultaneously. To compare model complexity across λ, we replace the parameter count with a matrix-valued generalization.

Definition 2.22 (Ridge hat matrix; effective degrees of freedom)**.** The **ridge hat matrix** is

$$H_\lambda := X(X^\top X + n\lambda I_p)^{-1}X^\top \in \mathbb{R}^{n\times n}, \qquad \hat{y}_\lambda = H_\lambda y.$$

The **effective degrees of freedom** is $\mathrm{df}(\lambda) := \mathrm{tr}(H_\lambda)$, where $\mathrm{tr}(M) := \sum_i M_{ii}$ is the **trace** (sum of diagonal entries) of a square matrix M.

Lemma 2.23 (Cyclic property of trace)**.** For any $A \in \mathbb{R}^{m\times n}$ and $B \in \mathbb{R}^{n\times m}$, $\mathrm{tr}(AB) = \mathrm{tr}(BA)$.

Proof. $\mathrm{tr}(AB) = \sum_i (AB)_{ii} = \sum_{i,k} A_{ik}B_{ki} = \sum_{k,i} B_{ki}A_{ik} = \sum_k (BA)_{kk} = \mathrm{tr}(BA)$. □

For OLS ($\lambda = 0$ under A2), H_λ reduces to P_X (Definition 2.13) and Lemma 2.23 gives

$$\mathrm{df}(0) = \mathrm{tr}(X(X^\top X)^{-1}X^\top) = \mathrm{tr}((X^\top X)^{-1}X^\top X) = \mathrm{tr}(I_p) = p,$$

recovering the familiar count of fitted parameters.

Theorem 2.24 (Ridge effective degrees of freedom)**.**

$$\mathrm{df}(\lambda) \;=\; \sum_{j=1}^{p} \frac{\sigma_j^2}{\sigma_j^2 + n\lambda}.$$

At $\lambda = 0$, $\mathrm{df}(0) = \mathrm{rank}(X)$ (so p under A2). As $\lambda \to \infty$, $\mathrm{df}(\lambda) \to 0$. The map $\lambda \mapsto \mathrm{df}(\lambda)$ is smooth and strictly monotone-decreasing on $[0, \infty)$ whenever X has at least one nonzero singular value.

Proof. Apply Lemma 2.23 with $A = X(X^\top X + n\lambda I_p)^{-1}$ and $B = X^\top$:

$$\mathrm{tr}(H_\lambda) \;=\; \mathrm{tr}\big(X^\top X (X^\top X + n\lambda I_p)^{-1}\big).$$

Substitute $X^\top X = V \Lambda V^\top$ and $(X^\top X + n\lambda I_p)^{-1} = V(\Lambda + n\lambda I_p)^{-1} V^\top$; applying $V^\top V = I_p$ and Lemma 2.23 again,

$$\mathrm{tr}(H_\lambda) \;=\; \mathrm{tr}\big(\Lambda(\Lambda + n\lambda I_p)^{-1}\big) \;=\; \sum_{j} \frac{\sigma_j^2}{\sigma_j^2 + n\lambda}.$$

At $\lambda \downarrow 0$, each summand with $\sigma_j > 0$ tends to 1; summands with $\sigma_j = 0$ are 0 throughout. Hence $\mathrm{df}(0) = \#\{j : \sigma_j > 0\} = \mathrm{rank}(X)$. As $\lambda \to \infty$, each summand $\to 0$. Each summand has derivative $-n\sigma_j^2/(\sigma_j^2 + n\lambda)^2 < 0$ when $\sigma_j > 0$; strict monotonicity follows. □

Interpretation. Ridge produces a continuous family of fits indexed by $\lambda \in [0, \infty)$, interpolating between full OLS ($\mathrm{df} = \mathrm{rank}(X)$) and the zero predictor ($\mathrm{df} = 0$). The map $\lambda \mapsto \mathrm{df}(\lambda)$ is monotone and smooth, so practitioners often parameterize ridge by df directly: the λ yielding $\mathrm{df}(\lambda) = p/2$ sits halfway between OLS and zero in complexity.

2.4 Bias–variance tradeoff

Ridge trades a controllable deterministic bias for a reduction in stochastic variance. OLS is unbiased under A3 (Theorem 2.11); ridge biases toward zero, paying off when the variance reduction outweighs the squared bias. The next theorems quantify both terms.

Theorem 2.25 (Bias and variance of ridge)**.** Let $M_\lambda := (X^\top X + n\lambda I_p)^{-1}$. Under A1–A4,

conditional on X:

$$\begin{aligned} \mathsf{E}[\hat\beta_\lambda \mid X] - \beta &= -n\lambda\, M_\lambda\, \beta, \\ \operatorname{var}(\hat\beta_\lambda \mid X) &= \sigma^2\, M_\lambda\, X^\top X\, M_\lambda \;=\; \sigma^2 (M_\lambda - n\lambda\, M_\lambda^2). \end{aligned}$$

In the eigenbasis V of $X^\top X$, writing $\beta_j := v_j^\top \beta$,

$$v_j^\top \big(\mathsf{E}[\hat\beta_\lambda \mid X] - \beta\big) \;=\; -\frac{n\lambda\, \beta_j}{\sigma_j^2 + n\lambda}, \qquad v_j^\top \operatorname{var}(\hat\beta_\lambda \mid X)\, v_j \;=\; \frac{\sigma^2\, \sigma_j^2}{(\sigma_j^2 + n\lambda)^2}.$$

Proof. Substitute $y = X\beta + \varepsilon$ into $\hat\beta_\lambda = M_\lambda X^\top y$:

$$\hat\beta_\lambda \;=\; M_\lambda X^\top X \beta + M_\lambda X^\top \varepsilon.$$

Conditional on X, A3 gives $\mathsf{E}[\varepsilon \mid X] = 0$, so $\mathsf{E}[\hat\beta_\lambda \mid X] = M_\lambda X^\top X\, \beta$. From $(X^\top X + n\lambda I_p) M_\lambda = I_p$ we get $X^\top X M_\lambda = I_p - n\lambda M_\lambda$; commuting $X^\top X$ and M_λ (both diagonal in V) gives $M_\lambda X^\top X = I_p - n\lambda M_\lambda$. Hence

$$\mathsf{E}[\hat\beta_\lambda \mid X] - \beta \;=\; -n\lambda\, M_\lambda\, \beta.$$

For the variance, apply Lemma 2.14 with $B = M_\lambda X^\top$ and A4:

$$\operatorname{var}(\hat\beta_\lambda \mid X) \;=\; \sigma^2\, M_\lambda X^\top X M_\lambda \;=\; \sigma^2 (M_\lambda - n\lambda\, M_\lambda^2).$$

The eigenbasis expressions follow from $M_\lambda v_j = (\sigma_j^2 + n\lambda)^{-1} v_j$. □

Theorem 2.26 (Ridge strictly improves OLS for some $\lambda > 0$)**.** Under A1–A4 and A2 (so all $\sigma_j > 0$), define the conditional MSE

$$\operatorname{MSE}(\lambda) \;:=\; \mathsf{E}[\|\hat\beta_\lambda - \beta\|^2 \mid X] \;=\; \|\,\mathsf{E}[\hat\beta_\lambda \mid X] - \beta\|^2 + \operatorname{tr}\big(\operatorname{var}(\hat\beta_\lambda \mid X)\big).$$

Then MSE is differentiable on $[0,\infty)$ with $\operatorname{MSE}'(0) = -2n\sigma^2 \sum_j \sigma_j^{-4} < 0$. Consequently $\operatorname{MSE}(\lambda) < \operatorname{MSE}(0)$ for all sufficiently small $\lambda > 0$: ridge strictly improves on OLS in MSE for some positive λ.

Proof. By Theorem 2.25, summing over the eigenbasis,

$$\mathrm{MSE}(\lambda) \;=\; \sum_{j=1}^{p} \frac{(n\lambda)^2\beta_j^2 + \sigma^2\sigma_j^2}{(\sigma_j^2 + n\lambda)^2}.$$

Differentiating the j-th summand at $\lambda = 0$ gives $-2n\sigma^2/\sigma_j^4$; summing,

$$\mathrm{MSE}'(0) \;=\; -2n\sigma^2 \sum_{j=1}^{p} \sigma_j^{-4} \;<\; 0.$$

By continuity, $\mathrm{MSE}(\lambda) < \mathrm{MSE}(0)$ for all sufficiently small $\lambda > 0$. □

Caveats. The MSE-optimal $\lambda^\star$ depends on the unknown β (through the β_j), the unknown σ^2, and the eigenvalue structure of $X^\top X$, so $\lambda^\star$ cannot be computed from data alone. Practitioners select λ by cross-validation (Unit 4); the theorem only guarantees that some positive λ is theoretically beneficial. The result also requires A2: when A2 fails, OLS is undefined, and ridge is the only option.

2.5 Standardization is non-negotiable

OLS is invariant under non-singular column rescaling: if $\tilde{X} = XD$ for invertible diagonal $D = \mathrm{diag}(d_1, \ldots, d_p)$, then

$$\hat{\beta}^{\tilde{X}}_{\mathrm{OLS}} = (\tilde{X}^\top \tilde{X})^{-1} \tilde{X}^\top y = D^{-1}(X^\top X)^{-1} D^{-1} \cdot DX^\top y = D^{-1} \hat{\beta}^{X}_{\mathrm{OLS}},$$

so the fitted values $\tilde{X}\hat{\beta}^{\tilde{X}} = XDD^{-1}\hat{\beta}^{X} = X\hat{\beta}^{X}$ are unchanged: rescaling columns alters the coefficients but not the predictions. Ridge does not enjoy this invariance.

Theorem 2.27 (Ridge fit depends on column scales)**.** For ridge at the same $\lambda > 0$, the fitted values $X\hat{\beta}^{X}_{\lambda}$ and $\tilde{X}\hat{\beta}^{\tilde{X}}_{\lambda}$ generally differ when $\tilde{X} = XD$ and $D \neq I_p$.

Proof. Take $p = 1$, $X = (1, 1)^\top$, $y = (1, 0)^\top$, and $\tilde{X} = (c, c)^\top = X \cdot c$ for some $c > 0$, $c \neq 1$. Direct computation:

$$\hat{\beta}^{X}_{\lambda} = \frac{X^\top y}{X^\top X + n\lambda} = \frac{1}{2 + 2\lambda}, \qquad \hat{\beta}^{\tilde{X}}_{\lambda} = \frac{c}{2c^2 + 2\lambda},$$

using $n = 2$. The fitted value at any row of X is $1/(2 + 2\lambda)$; at any row of $\tilde{X}$ it is $c \cdot \hat{\beta}^{\tilde{X}}_{\lambda} =$

$c^2/(2c^2 + 2\lambda) = 1/(2 + 2\lambda c^{-2})$. These differ whenever $c \neq 1$ and $\lambda > 0$. □

The penalty $(\lambda/2)\|\beta\|^2$ implicitly assumes coefficients live on a common scale; rescaling column j by d_j changes the **effective** penalty on that coordinate from λ to λd_j^{-2}.

Practical consequence: standardize. Before fitting ridge, transform each column of X to mean zero and unit variance:

$$X_{ij} \leftarrow \frac{X_{ij} - \bar{X}_j}{s_j}, \qquad \bar{X}_j := \frac{1}{n}\sum_i X_{ij}, \quad s_j := \sqrt{\frac{1}{n}\sum_i (X_{ij} - \bar{X}_j)^2}.$$

With standardized columns, all coefficients live on a comparable unit-variance scale, so a single λ treats them uniformly.

Intercept handling. The intercept is excluded from the penalty: penalizing it would let the ridge fit depend on the location of y, violating **translation invariance**. A simple shift of the target (Celsius to Kelvin, log-prices in cents to dollars) would change predictions, which is unacceptable. Standard practice: center y (subtract $\bar{y}$) and X (subtract column means), drop the intercept column, fit $\hat{\beta}$ on the centered, standardized data.

Un-scaling for prediction. The coefficient vector returned by the algorithm lives on the standardized scale; before applying it to raw data, transform back:

$$\hat{\beta}_j^{\text{orig}} = \frac{\hat{\beta}_j}{s_j}, \qquad \hat{\beta}_0^{\text{orig}} = \bar{y} - \sum_{j=1}^{p} \bar{X}_j \, \hat{\beta}_j^{\text{orig}}.$$

The first formula reverses the column scaling; the second reconstructs the intercept on the original y-scale. `glmnet` returns coefficients already un-scaled by default; a custom solver must perform this transformation explicitly before predicting on new data.

Standardize columns, exclude the intercept from the penalty, un-scale before prediction: forgetting any of the three is the most common ridge misuse.

3 Lasso

Ridge shrinks every coefficient toward zero but leaves all p of them nonzero. Selection requires a penalty that yields exact zeros: the **Lasso** replaces ℓ_2 by ℓ_1.

3.1 The Lasso problem and ℓ_1 geometry

Definition 2.28 (Lasso regression)**.** For tuning parameter $\lambda \geqslant 0$, the **Lasso** estimator minimizes the average squared-error loss with an ℓ_1 penalty on β:

$$\hat{\beta}^{\mathrm{L}}_{\lambda} \;:=\; \arg\min_{\beta\in\mathbb{R}^p} R^{\mathrm{L}}_{\lambda}(\beta), \qquad R^{\mathrm{L}}_{\lambda}(\beta) \;:=\; \tfrac{1}{2n}\|y - X\beta\|^2 + \lambda\|\beta\|_1,$$

where $\|\beta\|_1 := \sum_{j=1}^{p} |\beta_j|$. At $\lambda = 0$ this recovers OLS; for λ large enough, $\hat{\beta}^{\mathrm{L}}_{\lambda} = 0$. The $1/(2n)$ scaling matches 2.17, keeping λ comparable across Ridge, Lasso, and Elastic Net (and matching `glmnet`).

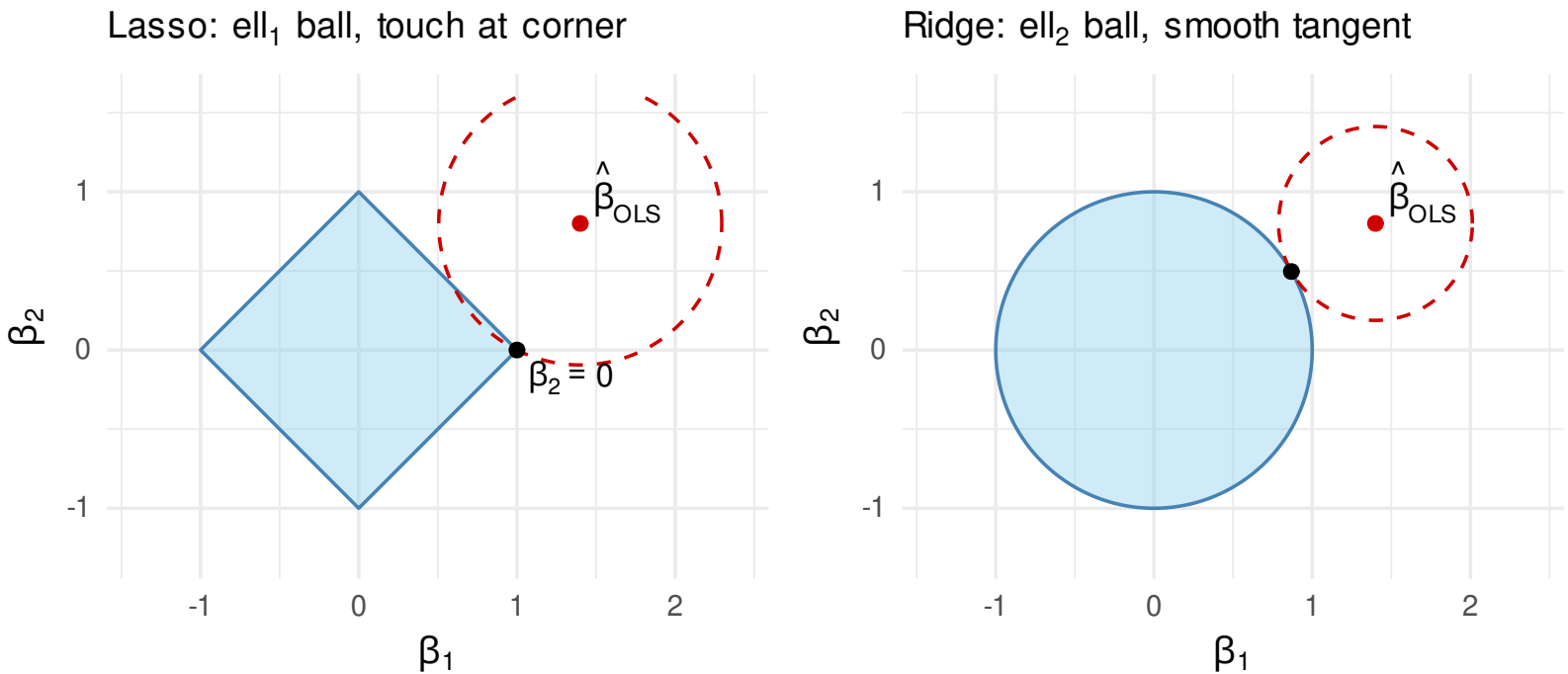
Ridge's ℓ_2 penalty is everywhere differentiable; the ℓ_1 penalty has kinks at $\beta_j = 0$. The kinks are the source of sparsity: the optimum can sit at one, fixing β_j to exactly 0. The next subsection makes this precise via subgradients and KKT.

Constrained reformulation. For every $\lambda > 0$ there is $t = t(\lambda) > 0$ with

$$\hat{\beta}^{\mathrm{L}}_{\lambda} \;=\; \arg\min_{\|\beta\|_1 \leqslant t} \|y - X\beta\|^2.$$

This is the standard Lagrangian duality for convex problems (Unit 6 develops the full theory). The constrained form makes the geometry transparent: minimize the squared-error contour over the ℓ_1 ball $B_1(t) := \{\beta : \|\beta\|_1 \leqslant t\}$.

ℓ_1 ball vs ℓ_2 ball. In $\mathbb{R}^p$, the ℓ_1 ball is a polytope with $2p$ vertices on the coordinate axes; the ℓ_2 ball is a sphere with no vertices. Each axis-aligned vertex of $B_1(t)$ has $p-1$ coordinates equal to zero.



The OLS objective has elliptical (here circular for clarity) contours centered at $\hat{\beta}_{\text{OLS}}$. The constrained minimum is the smallest contour that still touches the ball.

Remark (Why corners ⇒ sparsity)**.** When $\hat{\beta}_{\mathrm{OLS}}$ lies outside the ℓ_1 ball, the contact point is generically a vertex of $B_1(t)$, which has $p-1$ zero coordinates. The same construction with $B_2(t)$ produces a tangency on the smooth sphere, where all coordinates are generically nonzero. Lasso's sparsity is a corner phenomenon, not a shrinkage one. The next subsection makes this precise via subgradient calculus.

3.2 Subgradient and KKT conditions

R_λ^{L} is convex (sum of a convex quadratic and the convex ℓ_1 norm) but non-differentiable at any β with a zero coordinate. The **subgradient** generalizes the gradient to such points; Lasso optimality reads cleanly via subgradient calculus.

Definition 2.29 (Subdifferential of $|\cdot|$)**.** The **subdifferential** of the absolute-value function at $x \in \mathbb{R}$ is

$$\partial|x| = \begin{cases} \{+1\} & x > 0, \\ \{-1\} & x < 0, \\ [-1, 1] & x = 0. \end{cases}$$

A vector $g \in \mathbb{R}^p$ is a **subgradient** of $\beta \mapsto \|\beta\|_1$ at β if $g_j \in \partial|\beta_j|$ for every j.

Theorem 2.30 (Lasso KKT optimality)**.** $\hat\beta \in \mathbb{R}^p$ minimizes R^{L}_λ if and only if there exists a subgradient g of $\|\cdot\|_1$ at $\hat\beta$ with

$$\tfrac{1}{n} X_j^\top (y - X\hat\beta) \;=\; \lambda\, g_j, \qquad g_j \in \partial|\hat\beta_j|, \qquad j = 1, \dots, p.$$

Proof. R^{L}_λ is convex; the smooth term has gradient $\nabla[\frac{1}{2n}\|y - X\beta\|^2] = -\frac{1}{n}X^\top(y - X\beta)$, and the non-smooth term has subdifferential $\lambda\,\partial\|\beta\|_1$. The convex sum rule gives

$$\partial R^{\mathrm{L}}_\lambda(\beta) \;=\; -\tfrac{1}{n}X^\top(y - X\beta) + \lambda\,\partial\|\beta\|_1.$$

Fermat's rule: $\hat\beta$ minimizes R^{L}_λ iff $0 \in \partial R^{\mathrm{L}}_\lambda(\hat\beta)$, equivalently $\frac{1}{n}X^\top(y - X\hat\beta) = \lambda g$ for some $g \in \partial\|\hat\beta\|_1$. □

Unpacking $\partial|\hat\beta_j|$ from 2.29, the optimality system splits into two cases:

$$\tfrac{1}{n} X_j^\top (y - X\hat\beta) \;=\; \lambda\,\mathrm{sign}(\hat\beta_j) \quad \text{when } \hat\beta_j \neq 0; \qquad \left|\tfrac{1}{n} X_j^\top (y - X\hat\beta)\right| \;\leqslant\; \lambda \quad \text{when } \hat\beta_j = 0.$$

Lemma 2.31 (Active/inactive thresholding)**.** Let $\hat{\beta}$ be a Lasso minimizer and $r := y - X\hat{\beta}$. Then

$$\left|\tfrac{1}{n}X_j^\top r\right| \leqslant \lambda \quad \text{for all } j; \qquad \left|\tfrac{1}{n}X_j^\top r\right| < \lambda \;\Rightarrow\; \hat{\beta}_j = 0.$$

Proof. By Theorem 2.30, $\frac{1}{n}X_j^\top r = \lambda g_j$ with $g_j \in [-1,1]$, so $|\frac{1}{n}X_j^\top r| \leqslant \lambda$. If $|\frac{1}{n}X_j^\top r| < \lambda$ but $\hat{\beta}_j \neq 0$, then $g_j \in \{\pm 1\}$, forcing $|\frac{1}{n}X_j^\top r| = \lambda$, a contradiction. □

Remark. $\frac{1}{n}X_j^\top r$ is the empirical correlation between feature j and the current residual (after standardization, exactly the Pearson correlation). Below threshold λ, the feature carries less signal than the cost of activating it, and Lasso drives $\hat{\beta}_j$ to exactly 0.

3.3 Soft-thresholding: the orthonormal case

For X with orthonormal columns, the Lasso decouples across coordinates and admits a closed form. The result is the building block for coordinate descent (§3.4).

Definition 2.32 (Soft-thresholding operator)**.** For $a \in \mathbb{R}$ and $\lambda \geqslant 0$,

$$S_\lambda(a) \;:=\; \operatorname{sign}(a)\,\max(|a| - \lambda,\, 0) \;=\; \begin{cases} a - \lambda & a > \lambda, \\ 0 & |a| \leqslant \lambda, \\ a + \lambda & a < -\lambda. \end{cases}$$

S_λ shrinks a toward 0 by exactly λ when $|a| > \lambda$ and clips to 0 otherwise.

Theorem 2.33 (Lasso under orthonormal design)**.** If $\frac{1}{n}X^\top X = I_p$ (columns scaled so $\frac{1}{n}X_j^\top X_j = 1$, mutually orthogonal), then for each j,

$$\hat\beta_j^{\mathrm{L}} \;=\; S_\lambda\big(\tfrac{1}{n}X_j^\top y\big).$$

Proof. Under $\frac{1}{n}X^\top X = I_p$,

$$R_\lambda^{\mathrm{L}}(\beta) \;=\; \tfrac{1}{2n}\big(\|y\|^2 - 2\beta^\top X^\top y + \beta^\top X^\top X\beta\big) + \lambda\|\beta\|_1 \;=\; \tfrac{1}{2n}\|y\|^2 + \sum_{j=1}^{p}\big[\tfrac{1}{2}\beta_j^2 - a_j\beta_j + \lambda|\beta_j|\big],$$

with $a_j := \frac{1}{n}X_j^\top y$. The minimization separates across j. For each j, minimize $f(b) := \frac{1}{2}b^2 - a_j b + \lambda|b|$. Three cases:

- $b > 0$: $f'(b) = b - a_j + \lambda = 0 \Rightarrow b = a_j - \lambda$, valid iff $a_j > \lambda$.
- $b < 0$: $f'(b) = b - a_j - \lambda = 0 \Rightarrow b = a_j + \lambda$, valid iff $a_j < -\lambda$.
- $b = 0$: subgradient condition $-a_j + \lambda g = 0$ with $g \in [-1, 1]$, i.e. $|a_j| \leqslant \lambda$.

Combining gives $\hat\beta_j = S_\lambda(a_j)$. □

Remark. Compare ridge under the same design: $\hat\beta_j^{\mathrm{R}} = a_j/(1+\lambda)$, proportional shrinkage on every coordinate. Lasso's $S_\lambda(a_j)$ kills small a_j exactly and shrinks the survivors by a constant λ. The two estimators differ qualitatively: ridge softens; Lasso selects.

3.4 Coordinate descent

Real designs are not orthonormal. The standard fit algorithm exploits the per-coordinate result of 2.33 via **coordinate descent**: cycle through coordinates, optimizing one β_j at a time with the others held fixed. Each update is a soft-threshold.

Theorem 2.34 (Coordinate-wise Lasso update)**.** Fix $\beta_{-j} := (\beta_1, \dots, \beta_{j-1}, \beta_{j+1}, \dots, \beta_p)$ and let $r_{-j} := y - \sum_{k \neq j} X_k \beta_k$ be the partial residual excluding coordinate j. The minimizer in β_j alone is

$$\hat{\beta}_j \;=\; \frac{S_\lambda\big(\frac{1}{n} X_j^\top r_{-j}\big)}{\frac{1}{n} X_j^\top X_j}.$$

Proof. With β_{-j} fixed, R^{L}_λ as a function of β_j alone is (after expanding the square and dropping additive constants)

$$\tfrac{1}{2n}(X_j^\top X_j)\,\beta_j^2 - \tfrac{1}{n}(X_j^\top r_{-j})\,\beta_j + \lambda|\beta_j|.$$

Setting $c := \frac{1}{n} X_j^\top X_j$ and $a := \frac{1}{n} X_j^\top r_{-j}$ reduces to $\frac{c}{2}\beta_j^2 - a\beta_j + \lambda|\beta_j|$; the case analysis of 2.33 gives $\hat{\beta}_j = S_\lambda(a)/c$. □

Remark. After standardization (§2.5), $\frac{1}{n}X_j^\top X_j = 1$ for every j, and the update collapses to

$$\hat\beta_j \;=\; S_\lambda\big(\tfrac{1}{n}X_j^\top r_{-j}\big).$$

This is the form `glmnet` ships: λ is comparable across coordinates, and the threshold matches the value reported by the `cv.glmnet` object directly.

Theorem 2.35 (Coordinate-descent convergence)**.** Let $\beta^{(0)} \in \mathbb{R}^p$. The cyclic coordinate-descent iterates of 2.34 converge to a Lasso minimizer.

Proof. Write $f(\beta) := R^{\mathrm{L}}_\lambda(\beta) = g(\beta) + h(\beta)$ where $g(\beta) = \frac{1}{2n}\|y - X\beta\|^2$ is convex differentiable and $h(\beta) = \lambda\|\beta\|_1 = \sum_{j=1}^p \lambda|\beta_j|$ is convex and **separable** in the coordinates.
Step 1: monotone descent. Each cyclic update minimizes f exactly along one coordinate (2.34), so $f(\beta^{(k+1)}) \leqslant f(\beta^{(k)})$ for every k. The sequence $f(\beta^{(k)})$ is bounded below by 0, so $f^\star := \lim_k f(\beta^{(k)})$ exists.
Step 2: bounded iterates. On the level set $L := \{\beta : f(\beta) \leqslant f(\beta^{(0)})\}$, $\lambda\|\beta\|_1 \leqslant f(\beta^{(0)})$, so L is bounded. Each $\beta^{(k)} \in L$.

Step 3: a limit point exists. By Bolzano–Weierstrass, $\{\beta^{(k)}\}$ has a convergent subsequence $\beta^{(k_l)} \to \beta^\star$.

Step 4: $\beta^\star$ satisfies KKT. Suppose $\beta^\star$ violates 2.30 at coordinate j. The soft-threshold update for coordinate j (2.34) is continuous in the other coordinates; at $\beta^\star$ it yields a new value $v \neq \beta^\star_j$ with strict drop $\delta := f(\beta^\star) - f(\beta^\star_{[j\to v]}) > 0$. By continuity, the drop is at least $\delta/2$ at every $\beta^{(k_l)}$ for large l, so $f(\beta^{(k_l+1)}) \leqslant f(\beta^{(k_l)}) - \delta/2$ infinitely often, contradicting $f(\beta^{(k)}) \to f^\star$. Hence $\beta^\star$ satisfies 2.30 and minimizes f.

Step 5: full sequence converges. A second limit point would also satisfy 2.30 with the same active set $\mathcal{A}$ (under the genericity assumption used in Theorem 2.36; without it, distinct limit points may have different active sets but identical objective values), hence the same KKT system; strict convexity of g on $\mathrm{col}(X_\mathcal{A})$ makes the system's solution unique, so the two limits coincide. □

Remark. Separability is essential: cyclic CD on ℓ_1 converges, but a non-separable penalty like $\|A\beta\|_1$ with non-diagonal A can stall away from the optimum. `glmnet` adds active-set strategies, warm starts along the λ-path, and strong screening rules; runtime drops from $O(np)$ per pass to near-$O(n|\mathcal{A}|)$ for sparse paths.

3.5 Sparsity, paths, and Lasso vs. Ridge

The Lasso solution $\hat{\beta}^{\mathrm{L}}_{\lambda}$ traces a continuous, piecewise-linear path in β as λ varies; coordinates enter and leave the active set at specific knots determined by 2.31.

Theorem 2.36 (Piecewise linearity of the Lasso path)**.** The map $\lambda \mapsto \hat{\beta}^{\mathrm{L}}_{\lambda}$ is continuous and piecewise linear in λ on $(0, \infty)$. The active set $\mathcal{A}(\lambda) := \{j : \hat{\beta}^{\mathrm{L}}_{\lambda,j} \neq 0\}$ is constant on each linear piece, changing only at finitely many knots $\lambda_1 > \lambda_2 > \cdots$.

Proof. Fix $\lambda_0 > 0$ and let $\mathcal{A} := \{j : \hat{\beta}^{\mathrm{L}}_{\lambda_0,j} \neq 0\}$, $s_j := \operatorname{sign}(\hat{\beta}^{\mathrm{L}}_{\lambda_0,j})$ for $j \in \mathcal{A}$. Assume $X_{\mathcal{A}}$ has full column rank (genericity; holds outside a measure-zero set of designs and is the standard condition for Lasso uniqueness).

Affine on each piece. By the case-split form of 2.30 for active coordinates,

$$\tfrac{1}{n} X_{\mathcal{A}}^{\top}\big(y - X_{\mathcal{A}}\,\beta_{\mathcal{A}}\big) \;=\; \lambda\, s_{\mathcal{A}}.$$

Since $X_{\mathcal{A}}^{\top} X_{\mathcal{A}}$ is invertible,

$$\beta_{\mathcal{A}}(\lambda) \;=\; \big(X_{\mathcal{A}}^{\top} X_{\mathcal{A}}\big)^{-1}\big(X_{\mathcal{A}}^{\top} y - n\lambda\, s_{\mathcal{A}}\big),$$

which is affine in λ (off-active coordinates remain 0).

Locally constant active set. On a neighborhood of λ_0 where (a) $\operatorname{sign}(\beta_{\mathcal{A}}(\lambda)) = s_{\mathcal{A}}$ and (b) every inactive coordinate satisfies $|\frac{1}{n} X_j^\top (y - X_{\mathcal{A}}\, \beta_{\mathcal{A}}(\lambda))| < \lambda$, the formula above gives a Lasso minimizer (it satisfies 2.30); by the genericity assumption stated above, the Lasso minimizer is unique, so $\hat{\beta}^{\mathrm{L}}_\lambda$ is piecewise linear on this neighborhood.

Knots. The active set or sign vector changes at λ exactly when one of (a) (b) becomes a tight equality:

- **Departure**: an active coefficient $\beta_{\mathcal{A},j}(\lambda)$ crosses 0. Since $\beta_{\mathcal{A},j}(\lambda)$ is affine in λ, this happens at a single λ value per active coordinate.
- **Entry**: an inactive correlation $|\frac{1}{n} X_j^\top r(\lambda)|$ rises to λ. Both sides are affine in λ on the current piece, so the equality holds at a single λ value per inactive coordinate.

Each piece thus terminates at a single λ (the next knot below). Sweeping λ down from $\lambda_{\max} := \max_j |\frac{1}{n} X_j^\top y|$ (where $\hat{\beta} = 0$, by 2.30) to 0 generates finitely many pieces.

Continuity. At a knot, $\beta_{\mathcal{A}}(\lambda)$ from the upper piece matches $\beta_{\mathcal{A}'}(\lambda)$ from the lower piece on the intersection of the two active sets (both formulas reduce to the same KKT system), and the

changing coefficient is exactly zero at the knot in both pieces. □

Lemma 2.37 (Active-set cap)**.** For any $\lambda > 0$, assume $X_{\mathcal{A}(\lambda)}$ has full column rank (genericity, as in Theorem 2.36). Then $|\mathcal{A}(\lambda)| \leqslant \min(n, p)$.

Proof. The genericity assumption gives $X_{\mathcal{A}}^{\top} X_{\mathcal{A}}$ invertible, so $\operatorname{rank}(X_{\mathcal{A}}) = |\mathcal{A}|$. Since $\operatorname{rank}(X_{\mathcal{A}}) \leqslant \min(n, p)$, we have $|\mathcal{A}| \leqslant \min(n, p)$. □

Remark.

- **Ridge vs. Lasso.** Ridge shrinks every coordinate continuously; the path is everywhere differentiable in λ and never produces exact zeros. Lasso has piecewise-linear paths with kinks at knots; coordinates take values exactly 0 on long intervals.
- **Variable-selection cap.** 2.37 is the source of Lasso's $p > n$ behavior: the Lasso can select at most n variables. With $p \gg n$ (genomics, text features), this is a hard ceiling; if the truth has more than n relevant features, Lasso cannot recover all of them. The Elastic Net (§4) lifts this cap.
- **Where Ridge wins.** Ridge handles collinearity via spectral lift (§2.2) but does not select. When the analyst wants stable predictions and does not need a sparse model (e.g. all features genuinely matter), ridge is the right tool.

4 Elastic Net

4.1 Motivation: Lasso under correlated features

When two features are highly correlated, Lasso picks one and zeros the other, and the choice flips on small noise perturbations. The Elastic Net adds an ℓ_2 term to the penalty, stabilizing the choice and

lifting the $|\mathcal{A}| \leqslant n$ cap.

Example 2.1 (Lasso instability under perfect collinearity)**.** Let $X_1 = X_2$ exactly and consider the Lasso with $\lambda > 0$. By Theorem 2.30, any $\hat{\beta}$ with $\hat{\beta}_1 + \hat{\beta}_2 = c^\star$ (some fixed value) and matching signs satisfies the KKT conditions; the partition of $c^\star$ between $\hat{\beta}_1$ and $\hat{\beta}_2$ is undetermined. With $X_1 \approx X_2$ (near-collinear), the same instability holds approximately: tiny noise perturbations produce wildly different selections.

4.2 Elastic Net definition and the grouping effect

Definition 2.38 (Elastic Net)**.** For $\lambda \geqslant 0$ and mixing parameter $\alpha \in [0, 1]$,

$$\hat{\beta}_{\lambda,\alpha} \;:=\; \arg\min_{\beta \in \mathbb{R}^p} \tfrac{1}{2n}\|y - X\beta\|^2 + \lambda\big[\alpha\|\beta\|_1 + \tfrac{1-\alpha}{2}\|\beta\|^2\big].$$

$\alpha = 1$ recovers Lasso; $\alpha = 0$ recovers ridge (with the same $1/(2n)$ scaling as 2.17). Intermediate α blends ℓ_1-sparsity with ℓ_2-stability. The form matches `glmnet` exactly.

Lemma 2.39 (Grouping effect)**.** For perfectly correlated features $X_j = X_k$ and Elastic Net with $\alpha < 1$, the unique minimizer satisfies $\hat{\beta}_j = \hat{\beta}_k$.

Proof. With $X_j = X_k$, the data fit depends on $\beta_j + \beta_k$ alone. The penalty $\alpha(|\beta_j| + |\beta_k|) + \frac{1-\alpha}{2}(\beta_j^2 + \beta_k^2)$ subject to $\beta_j + \beta_k = c$ is minimized at $\beta_j = \beta_k = c/2$ when $\alpha < 1$ (the strict convexity of the squared term breaks the indifference). The optimum is unique, with $\hat{\beta}_j = \hat{\beta}_k$. □

Remark.

- **No active-set cap.** The ℓ_2 component restores invertibility of $X_{\mathcal{A}}^\top X_{\mathcal{A}} + \lambda(1-\alpha)I$ (cf. 2.18); the $\min(n, p)$ ceiling on $|\mathcal{A}|$ disappears.
- **Tuning.** α and λ are both selected by cross-validation (§5). Common starting choices: $\alpha = 0.5$ for moderate sparsity with stability; $\alpha \to 1$ when sparsity is the goal; $\alpha \to 0$ when collinearity dominates.
- **Coordinate update.** With the ℓ_2 term, the standardized coordinate-descent update becomes $\hat{\beta}_j = S_{\lambda\alpha}(\frac{1}{n} X_j^\top r_{-j})/(1 + \lambda(1-\alpha))$, an inexpensive modification to 2.34.

5 Selecting λ

5.1 Cross-validation

The penalty parameter λ is a tuning hyperparameter; estimating it from training error alone overfits ($\lambda = 0$ minimizes training error). The standard remedy is K-fold cross-validation.

Definition 2.40 (K-fold CV for λ)**.** Partition the training set into K folds. For each candidate λ in a grid and each fold $k \in \{1, \dots, K\}$:

- Fit $\hat{\beta}_\lambda^{(-k)}$ on the data with fold k removed.
- Evaluate $\mathrm{MSE}_k(\lambda) := |\mathcal{I}_k|^{-1} \sum_{i \in \mathcal{I}_k} (y_i - x_i^\top \hat{\beta}_\lambda^{(-k)})^2$ on fold k (index set $\mathcal{I}_k$).

The CV criterion is $\mathrm{CV}(\lambda) := K^{-1} \sum_k \mathrm{MSE}_k(\lambda)$. Two natural choices:

- $\hat{\lambda}_{\min} := \arg\min_\lambda \mathrm{CV}(\lambda)$ (best CV score).
- $\hat{\lambda}_{1\mathrm{se}} := \max\{\lambda : \mathrm{CV}(\lambda) \leqslant \mathrm{CV}(\hat{\lambda}_{\min}) + \mathrm{SE}(\hat{\lambda}_{\min})\}$ (one-standard-error rule: largest λ within 1 SE of the minimum).

Remark.

- **The 1-SE rule biases toward stronger regularization** (sparser models, more shrinkage): the minimum CV score has nontrivial standard error, and a slightly worse CV often gives a meaningfully simpler model. Report both. `glmnet` returns them as `lambda.min` and `lambda.1se`.
- **The CV grid is data-driven.** `glmnet` starts at $\lambda_{\max} := \max_j |\frac{1}{n} X_j^\top y|$ (the smallest λ giving $\hat\beta = 0$, by 2.30) and runs down to $\lambda_{\max} \cdot 10^{-3}$ on a geometric grid. Warm starts along the path exploit the piecewise-linear structure of 2.36: the solution at λ_k initializes the descent at λ_{k+1}.

5.2 Information criteria caveat

AIC and BIC penalize model complexity directly, an alternative to CV. For Lasso this requires an unbiased estimator of $\mathrm{df}(\lambda)$, the effective degrees of freedom (§2.3 for Ridge).

Lemma 2.41 (Lasso degrees of freedom)**.** Under standard regularity, $\mathsf{E}[\mathrm{df}(\hat\beta^{\mathrm{L}}_\lambda)] = \mathsf{E}[|\mathcal{A}(\lambda)|]$. The active-set size is an unbiased estimator of df.

Proof. Take $y \sim \mathcal{N}(\mu, \sigma^2 I_n)$ for the noise model. Define the effective degrees of freedom via the covariance form

$$\mathrm{df}(\hat{\mu}) \;:=\; \sigma^{-2} \sum_{i=1}^{n} \mathrm{cov}\big(\hat{\mu}_i, y_i\big).$$

Stein's identity (prerequisite). For $y \sim \mathcal{N}(\mu, \sigma^2 I_n)$ and a weakly differentiable $g : \mathbb{R}^n \to \mathbb{R}^n$ with finite second moments,

$$\sigma^{-2} \sum_{i=1}^{n} \mathrm{cov}\big(g_i(y), y_i\big) \;=\; \mathsf{E}\Big[\sum_{i=1}^{n} \tfrac{\partial g_i}{\partial y_i}(y)\Big] \;=\; \mathsf{E}\big[\mathrm{tr}\big(\partial g(y)/\partial y\big)\big].$$

This is integration by parts against the Gaussian density: $\partial_{y_i} \log \phi(y_i; \mu_i, \sigma^2) = -(y_i - \mu_i)/\sigma^2$, so $\sigma^{-2}\,\mathrm{cov}(g_i, y_i) = \sigma^{-2}\,\mathsf{E}[(y_i - \mu_i) g_i] = \mathsf{E}[\partial g_i / \partial y_i]$ when boundary terms vanish.

Lasso fitted values on an active piece. On any open set where $\mathcal{A}(\lambda; y)$ and s are constant, 2.36's formula gives

$$\hat{\mu}(y) \;=\; X_{\mathcal{A}}\, \hat{\beta}_{\mathcal{A}}(y) \;=\; X_{\mathcal{A}} \big(X_{\mathcal{A}}^{\top} X_{\mathcal{A}}\big)^{-1} X_{\mathcal{A}}^{\top} y \;-\; n\lambda\, X_{\mathcal{A}} \big(X_{\mathcal{A}}^{\top} X_{\mathcal{A}}\big)^{-1} s.$$

The Jacobian $\partial\hat{\mu}/\partial y$ equals the projection $P_{\mathrm{col}(X_{\mathcal{A}})} := X_{\mathcal{A}}(X_{\mathcal{A}}^\top X_{\mathcal{A}})^{-1}X_{\mathcal{A}}^\top$.

Trace of a projection. A projection onto a k-dimensional subspace has trace k (sum of k ones and $n-k$ zeros along its eigenvalues). Hence $\mathrm{tr}(\partial\hat{\mu}/\partial y) = |\mathcal{A}|$ on each active piece.

Combining. Stein's identity gives $\mathrm{df}(\hat{\mu}) = \mathsf{E}[\mathrm{tr}(\partial\hat{\mu}/\partial y)]$. The set of y with multiple knots is measure zero (a finite union of measure-zero hyperplanes from the entry/departure equations), so

$$\mathrm{df}(\hat{\mu}) \;=\; \mathsf{E}[|\mathcal{A}(\lambda)|].$$

□

Remark. With $\mathrm{df} \approx |\mathcal{A}(\lambda)|$, BIC for Lasso reads $\mathrm{BIC}(\lambda) = n\log\widehat{\mathrm{MSE}}(\lambda) + |\mathcal{A}(\lambda)|\log n$. BIC asymptotically selects the true sparse model when one exists; AIC tends to over-select. Use CV when the goal is predictive risk; use AIC/BIC when the goal is support recovery.

6 Implementation Tour

The theory of §§2–5 maps onto two production tools: `glmnet` (the canonical solver, the engine inside tidymodels) and the tidymodels unified API. A from-scratch coordinate-descent Lasso closes the loop

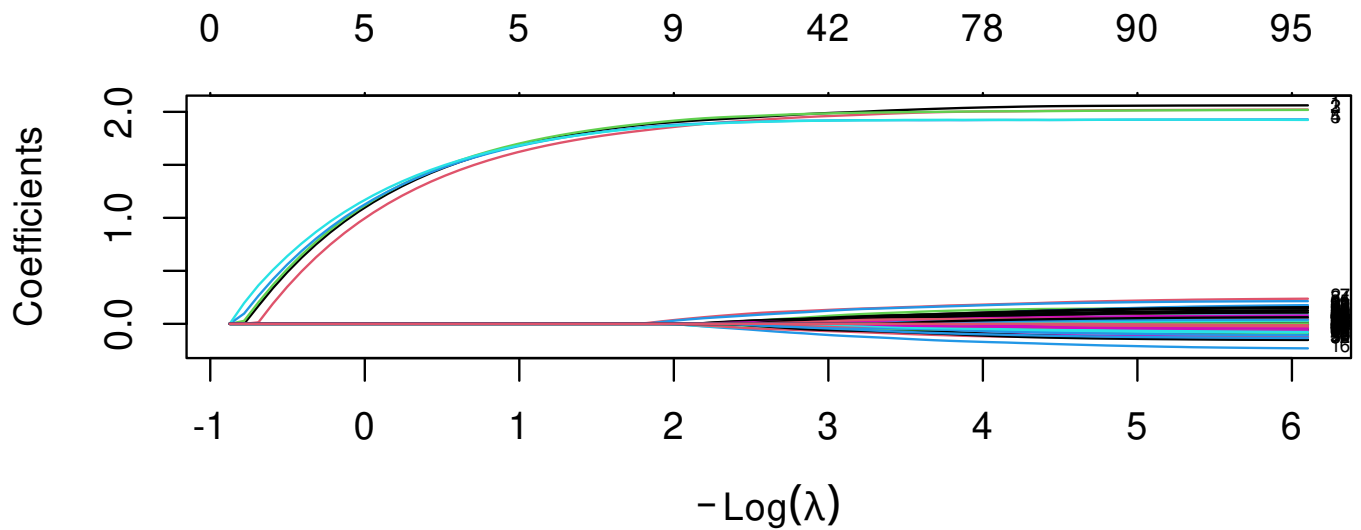
in twenty lines.

6.1 Glmnet workflow

`glmnet` fits Lasso, Ridge, and Elastic Net via cyclic coordinate descent along a λ-path. `cv.glmnet` performs K-fold CV automatically.

```r
library(glmnet)
set.seed(1)
n <- 200; p <- 100
X <- matrix(rnorm(n * p), n, p)
beta_true <- c(rep(2, 5), rep(0, p - 5))
y <- as.vector(X %*% beta_true + rnorm(n, sd = 1))

# Lasso (alpha = 1) along a default lambda grid; standardization is on by default
fit_lasso <- glmnet(X, y, alpha = 1)
plot(fit_lasso, xvar = "lambda", label = TRUE)        # coefficient paths
```




```r
# Cross-validated lambda
cv <- cv.glmnet(X, y, alpha = 1, nfolds = 10)
c(lambda_min = cv$lambda.min, lambda_1se = cv$lambda.1se)
```

```
## lambda_min lambda_1se
##  0.1013816  0.1614282
```

```r
# Active set at lambda.1se
nz <- which(coef(cv, s = "lambda.1se")[-1] != 0)       # drop intercept
nz                                                     # should contain {1,...,5}, likely
```

```
## [1]  1  2  3  4  5 27 84
```

Remark. `glmnet`'s loss matches 2.28, 2.17, and 2.38 term for term, so λ values reported by `cv.glmnet` are directly the λ of the theorems above. No rescaling is needed when comparing analytic results to fitted output.

6.2 Coordinate descent from scratch

The full Lasso solver in twenty lines, matching `glmnet` on standardized data.

```r
soft_thresh <- function(a, lambda) sign(a) * pmax(abs(a) - lambda, 0)

lasso_cd <- function(X, y, lambda, max_iter = 1000, tol = 1e-7) {
  n <- nrow(X); p <- ncol(X)
  Xs <- scale(X, center = TRUE, scale = TRUE) * sqrt(n / (n - 1))   # var = 1 with n-divi
  ys <- y - mean(y)
  beta <- rep(0, p)
```

```r
  r    <- as.vector(ys - Xs %*% beta)                                  # running full resi
  for (it in seq_len(max_iter)) {
    beta_old <- beta
    for (j in seq_len(p)) {
      r_j        <- r + Xs[, j] * beta[j]                              # add j back: O(n)
      beta_j_new <- soft_thresh(sum(Xs[, j] * r_j) / n, lambda)        # standardized upda
      r          <- r_j - Xs[, j] * beta_j_new                         # remove new j: O(n
      beta[j]    <- beta_j_new
    }
    if (max(abs(beta - beta_old)) < tol) break
  }
  list(beta = beta, iter = it, intercept = mean(y))
}

# Smoke test against glmnet (compare on standardized scale: glmnet stores
# beta on the original X scale, so multiply by n-divisor SD to match)
set.seed(1); n <- 200; p <- 50
X <- matrix(rnorm(n * p), n, p)
beta_true <- c(rep(1.5, 5), rep(0, p - 5))
y <- as.vector(X %*% beta_true + rnorm(n))
fit   <- lasso_cd(X, y, lambda = 0.05)
```

```r
fit_g <- glmnet::glmnet(X, y, alpha = 1, lambda = 0.05, standardize = TRUE)
sds_n <- apply(X, 2, sd) * sqrt((n - 1) / n)
max(abs(fit$beta - as.vector(fit_g$beta) * sds_n))                    # O(1e-5)
```

```
## [1] 2.618674e-05
```

Remark.

- $O(np)$ **per epoch via the running residual.** The residual r updates in $O(n)$ per coordinate (one add, one subtract); a full pass over p coordinates is $O(np)$, not $O(np^2)$. Production `glmnet` adds active-set strategies (skip coordinates zeroed in the previous pass), warm starts along the λ-path, and strong screening rules, cutting per-pass cost to near-$O(n|\mathcal{A}|)$ on sparse solutions.
- **Un-scaling for prediction.** `fit$beta` is on the standardized scale; to predict on raw new data, compute the population SD and column means directly from X (avoiding any dependence on `Xs`, which is local to the solver), and reconstruct $\hat{\beta}$ on the original scale:

```
s_pop          <- apply(X, 2, sd) * sqrt((nrow(X) - 1) / nrow(X))
beta_orig      <- fit$beta / s_pop
intercept_orig <- mean(y) - sum(colMeans(X) * beta_orig)
```

  The factor $\sqrt{(n-1)/n}$ converts R's sample SD (`sd()` uses $n-1$) to the population SD (n divisor) used inside `lasso_cd`. `glmnet` performs both transformations internally; a custom solver must do it explicitly.

6.3 Tidymodels integration

The same pipeline through `tidymodels`: a recipe (preprocessing, including standardization) feeds into `parsnip::linear_reg(penalty, mixture)`; `tune_grid` tunes both λ (`penalty`) and α (`mixture`) by CV.

```
library(tidymodels); library(glmnet)
set.seed(1)
df <- tibble::tibble(y = y, as.data.frame(X))          # X, y from previous chunk

split <- initial_split(df, prop = 0.8)
train <- training(split); test <- testing(split)

rec <- recipe(y ~ ., data = train) |>
  step_normalize(all_numeric_predictors())             # standardization in-recipe

mod <- linear_reg(penalty = tune(), mixture = tune()) |> set_engine("glmnet")
wf  <- workflow() |> add_recipe(rec) |> add_model(mod)

grid <- grid_regular(penalty(range = c(-4, 0)), mixture(range = c(0, 1)),
                     levels = c(20, 5))
```

```r
folds <- vfold_cv(train, v = 10)
res <- tune_grid(wf, resamples = folds, grid = grid, metrics = metric_set(rmse))

best     <- select_by_one_std_err(res, metric = "rmse", desc(penalty))
final_wf <- finalize_workflow(wf, best)

# Generalization estimate: refits on train, predicts on test, reports metrics
final_res <- last_fit(final_wf, split)
collect_metrics(final_res)
```

```
## # A tibble: 2 x 4
##   .metric .estimator .estimate .config
##   <chr>   <chr>          <dbl> <chr>
## 1 rmse    standard       0.870 pre0_mod0_post0
## 2 rsq     standard       0.940 pre0_mod0_post0
```

```r
# Deployment-ready fit on full training data (uncomment when needed)
# final_fit <- fit(final_wf, data = train)
```

Remark. Placing `step_normalize` inside the recipe (not as preprocessing on the full dataset) is the central convention: standardization parameters fit on each training fold, applied to its corresponding validation portion, never crossing the train/test boundary. §7.2 treats the leakage trap that arises when this convention is violated.

7 Pitfalls and Misapplications

Each entry pairs a misuse with a diagnostic and a runnable demo. Standardization, leakage, and instability dominate the list because Lasso/Ridge are sensitive to data preparation in ways OLS is not.

7.1 Forgetting to standardize

The penalty $\lambda\|\beta\|^2$ (Ridge) or $\lambda\|\beta\|_1$ (Lasso) treats every coordinate identically; rescaling column j by d_j changes the effective penalty on that coordinate from λ to λd_j^{-2} (§2.5). Mixing scales (income in dollars, age in years) without standardization arbitrary-weights the penalty across features.

```
library(glmnet)
set.seed(1); n <- 200; p <- 5
X <- matrix(rnorm(n * p), n, p)
X[, 1] <- X[, 1] * 1000                                  # blow up scale of column 1
```

```r
beta_true <- c(0.001, rep(1, p - 1))                    # corresponding small true coef
y <- as.vector(X %*% beta_true + rnorm(n))

# Ridge without standardization: column 1 is shrunk far more than the others
fit_no_std <- glmnet(X, y, alpha = 0, lambda = 1, standardize = FALSE)
fit_std    <- glmnet(X, y, alpha = 0, lambda = 1, standardize = TRUE)
cbind(no_std = as.vector(fit_no_std$beta),
      with_std = as.vector(fit_std$beta),
      truth = beta_true)
```

```
##             no_std     with_std truth
## [1,] 0.0009628367 0.0006919441 0.001
## [2,] 0.7134600359 0.7068647999 1.000
## [3,] 0.7280703242 0.7138701041 1.000
## [4,] 0.7457985559 0.7243309634 1.000
## [5,] 0.7155111164 0.6962291962 1.000
```

Diagnostic. Any column with non-unit variance after preprocessing breaks scale-invariance.

Fix. `glmnet`'s `standardize = TRUE` default; in tidymodels, `step_normalize(all_numeric_predi` in the recipe.

7.2 Preprocessing outside CV folds

Preprocessing parameters fit once on the full training set leak validation-fold information into every training-fold model. Standardization is the textbook case but its bias is mild for iid Lasso/Ridge; the principle becomes pedagogically dramatic when preprocessing is response-aware. Pick the top-20 features by their full-data $|\mathrm{cor}(X_j, y)|$, then 10-fold CV the resulting OLS. Below, y is pure noise.

```
library(tidymodels)
set.seed(2026); n <- 60; p <- 2000
X <- matrix(rnorm(n * p), n, p)
y <- rnorm(n)                                          # null signal
df_raw <- data.frame(y = y, X)

# WRONG: pre-screen by full-data |cor(X, y)|, then CV the resulting OLS.
# Each fold's training-portion model sees 20 features chosen with help from
# that fold's validation portion.
cors  <- abs(cor(X, y))[, 1]
top20 <- order(cors, decreasing = TRUE)[1:20]
df_w  <- data.frame(y = y, X[, top20])
mod_ols <- linear_reg() |> set_engine("lm")
set.seed(7)
```

```r
fit_wrong <- fit_resamples(
  workflow() |> add_recipe(recipe(y ~ ., df_w)) |> add_model(mod_ols),
  resamples = vfold_cv(df_w, v = 10), metrics = metric_set(rmse))

# RIGHT: keep all p features; Lasso selects inside each fold's training
# portion. Selection never sees the validation rows.
mod_lasso <- linear_reg(penalty = 0.1, mixture = 1) |> set_engine("glmnet")
set.seed(7)
fit_right <- fit_resamples(
  workflow() |> add_recipe(recipe(y ~ ., df_raw)) |> add_model(mod_lasso),
  resamples = vfold_cv(df_raw, v = 10), metrics = metric_set(rmse))

c(honest_baseline = sd(y),                                  # truth: no signal
  wrong_screen    = collect_metrics(fit_wrong)$mean,
  right_in_fold   = collect_metrics(fit_right)$mean)
```

```
## honest_baseline    wrong_screen   right_in_fold
##       0.9380661       0.5374912       0.9228452
```

The honest baseline is $\mathrm{sd}(y)$ since y is independent of X. Leakage roughly halves the WRONG CV-RMSE; the RIGHT pipeline recovers the truth.

Diagnostic. CV scores noticeably better than the naive $\hat{y} = \bar{y}_{\text{train}}$ baseline, or far better than a held-out test score, are the symptom.

Fix. Every preprocessing step lives inside the recipe (`step_normalize`, `step_corr`, `step_pca`, `step_dummy`, response-aware screening via `step_select_*` from `recipeselectors` or `colino`); `tune_grid` then refits the recipe per fold, isolating each fold's preprocessing parameters from its validation portion.

7.3 Lasso instability under collinearity

When two features are nearly collinear, Lasso picks one and zeros the other; the choice flips on small data perturbations (2.1). The reported active set is unstable.

```
set.seed(1); n <- 100
x1 <- rnorm(n); x2 <- x1 + rnorm(n, sd = 0.1)          # near-collinear
X  <- cbind(x1, x2, matrix(rnorm(n * 3), n, 3))
y  <- 1.0 * x1 + rnorm(n)                              # truth: only x1

# Refit on bootstrap samples; track which of x1, x2 is selected
set.seed(2)
selected <- replicate(100, {
```

```
  idx <- sample(n, replace = TRUE)
  fit <- glmnet(X[idx, ], y[idx], alpha = 1, lambda = 0.1, standardize = TRUE)
  c(x1 = fit$beta[1, 1] != 0, x2 = fit$beta[2, 1] != 0)
})
rowMeans(selected)                                    # x1 and x2 each appear < 100% of t
```

```
##   x1   x2
## 0.73 0.42
```

Diagnostic. Bootstrap stability of the active set; if a coefficient is selected in $< 70\%$ of bootstraps, treat its inclusion as noise.

Fix. Elastic Net with $\alpha < 1$ (2.38); the ℓ_2 component induces grouping (2.39) and stabilizes selection.

7.4 Lasso on a correlated group: arbitrary picks

A subtler version of §7.3: if features $X_1, \ldots, X_5$ are equally correlated with y and with each other, Lasso may pick any one (or a random subset); the others are zeroed despite being equally informative.

```
# Five equally informative correlated features
set.seed(1); n <- 200
z <- rnorm(n)
X <- sapply(1:5, function(j) z + rnorm(n, sd = 0.2)) # all near-z
```

```r
y <- z + rnorm(n)
fit_l  <- glmnet(X, y, alpha = 1,   lambda = 0.05, standardize = TRUE)
fit_en <- glmnet(X, y, alpha = 0.5, lambda = 0.05, standardize = TRUE)
cbind(lasso = as.vector(fit_l$beta), enet = as.vector(fit_en$beta))
```

```
##           lasso       enet
## [1,] 0.0000000 0.00000000
## [2,] 0.1968584 0.22268484
## [3,] 0.3149996 0.30792305
## [4,] 0.0000000 0.03116284
## [5,] 0.4276885 0.39838828
```

```r
# lasso: arbitrary subset retained with uneven sizes; enet: more spread, all
# correlated features carry comparable nonzero weight.
```

Diagnostic. Domain knowledge that a group is equally informative, conflicting with Lasso picking only one.

Fix. Elastic Net with $\alpha \in (0, 1)$, which by 2.39 assigns equal coefficients to perfectly correlated features and approximately so to near-correlated ones.

7.5 Lasso's $\min(n, p)$ active-set cap

2.37 bounds the Lasso active set by $\min(n, p)$. In high-dimensional settings ($p \gg n$, e.g. genomics with $p = 20{,}000$ genes and $n = 200$ patients), at most 200 features can be selected. If the truth involves more, Lasso cannot recover them.

```
# p > n; truth has 50 active coefficients but n = 30
set.seed(1); n <- 30; p <- 200
X <- matrix(rnorm(n * p), n, p)
beta_true <- c(rep(0.5, 50), rep(0, p - 50))
y <- as.vector(X %*% beta_true + rnorm(n))

fit <- glmnet(X, y, alpha = 1, lambda = 0.1, standardize = TRUE)
sum(fit$beta != 0)                                    # at most n, regardless of truth
```

```
## [1] 28
```

Diagnostic. Count the active-set size relative to n; if $|\mathcal{A}| = n$ at the chosen λ, the cap is binding.

Fix. Elastic Net with $\alpha < 1$ removes the cap (the ℓ_2 term restores the active block's invertibility); or pre-screening to reduce p below n.

7.6 Ridge does not perform selection

Ridge handles collinearity via spectral lift (§2.2), but every coefficient stays nonzero for any $\lambda < \infty$. Reporting "the most important variables" from Ridge by ranking $|\hat{\beta}_j|$ confuses shrinkage with selection.

```r
set.seed(1); n <- 200; p <- 20
X <- matrix(rnorm(n * p), n, p)
beta_true <- c(rep(2, 3), rep(0, p - 3))
y <- as.vector(X %*% beta_true + rnorm(n))

fit_ridge <- glmnet(X, y, alpha = 0, lambda = 0.5, standardize = TRUE)
sum(fit_ridge$beta == 0)                                  # 0: all coefs nonzero
```

```
## [1] 0
```

```r
fit_lasso <- glmnet(X, y, alpha = 1, lambda = 0.1, standardize = TRUE)
which(as.vector(fit_lasso$beta) != 0)                     # exact selection
```

```
## [1] 1 2 3
```

Diagnostic. Count zero coefficients; if all are nonzero, Ridge is shrinking, not selecting.

Fix. Lasso (2.28) or Elastic Net (2.38) when sparsity is the goal. Ridge coefficients are continuous shrinkage estimates; thresholding them for variable importance is a misuse.